Backend-for-Frontend (BFF)

one backend per client experience

Watch the 4-min video

In one line: a backend-for-frontend is a thin backend built for one client experience (web, mobile, partner), owned by that frontend's team, that aggregates the shared services and returns exactly the data the screen needs; business rules stay in the domain services.

1 One API that serves no one well

The web app shows rich dashboards. The mobile app needs tiny payloads over a slow network. A partner needs a stable, versioned API. One general-purpose API tries to serve all three: it grows optional fields, client-specific flags and special endpoints, and every change for one client risks breaking another. The **BFF** pattern (described by Sam Newman; popularised by SoundCloud and Netflix) gives each client experience its own backend.

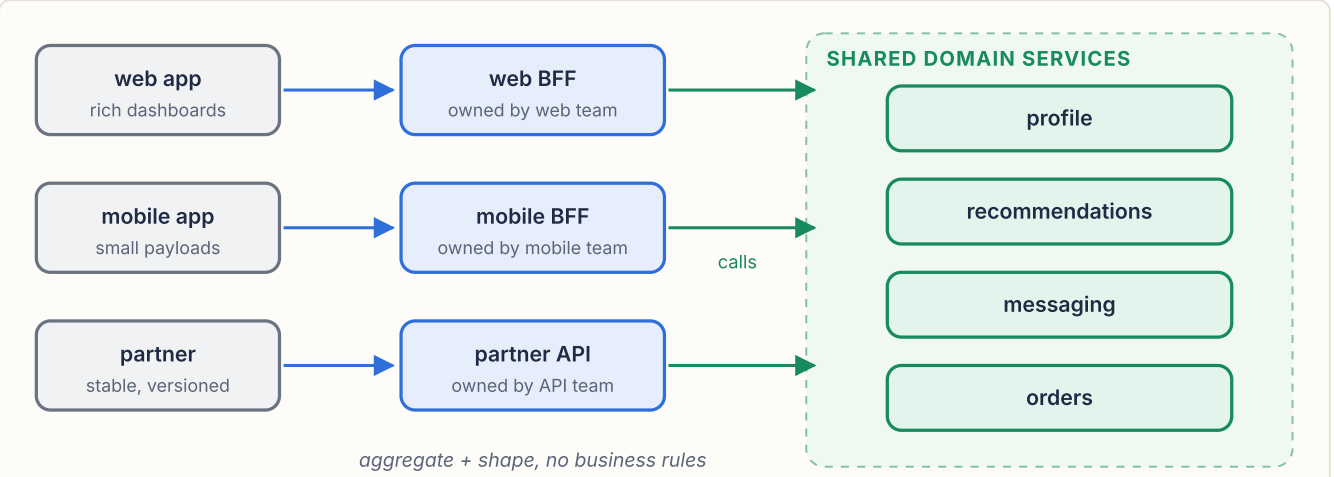


Figure 1. Each client gets its own thin backend, owned by the team that builds that client. All BFFs share the same domain services.

2 What a BFF does

- **Aggregates:** calls several services in parallel, server-side, where round trips take about a millisecond instead of 100 ms+ on mobile.
- **Shapes:** trims fields, flattens nested data, formats dates and money for the screen, and pages results the way the UI scrolls.
- **Adapts protocols:** e.g. gRPC to services, JSON or GraphQL to the client; handles client-specific auth flows and caching.

It is an API designed for **one** screen set, not for every possible consumer.

3 Ownership is the point

The mobile team owns the mobile BFF. When the home screen changes, they change the screen and its BFF and ship both together, with no ticket to a central gateway team and no risk to the web client. This matches Conway's Law (lesson 19): one team, one client, one backend.

	One general API / gateway	BFF per client
Payload	superset for every client	exactly what the screen needs
Change for one client	may break others	isolated to that BFF
Owner	central platform team	the frontend team
Number of backends	one	one per client type (more to run)
Risk	bloat, bottleneck team	duplicated logic across BFFs

BFFs and gateways work together: a thin gateway (lesson 25) in front still does TLS, token validation and rate limiting, then routes to the right BFF. **GraphQL** is the main alternative: one flexible endpoint where each client asks for exactly the fields it wants.

4 Costs

More services to build, deploy and monitor. Logic tends to get **duplicated** across BFFs ("is this order cancellable?" computed in web and mobile BFFs, differently). Keep such rules in the domain services and keep BFFs thin: aggregation and shaping only.

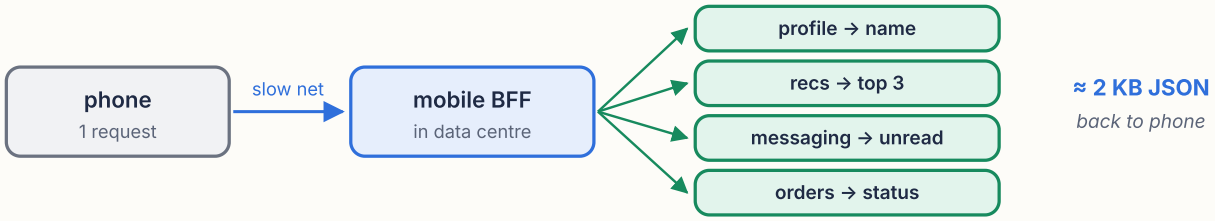


Figure 2. The mobile home screen: four calls inside the data centre, one compact response over the mobile network.

WORKED EXAMPLE: THE MOBILE HOME SCREEN

The screen shows the user's name, three recommendations, the unread message count and the current order status. The mobile BFF calls the profile, recommendation, messaging and order services **in parallel**, each with a short timeout. It keeps only the fields the screen shows and returns one payload of about **2 KB** instead of four full responses. If recommendations time out, it returns the rest with an empty list, so the screen still loads. Fewer bytes and one round trip mean faster loads and less battery and data use.

COMMON MISTAKES

- Business rules in BFFs, duplicated and drifting apart.
- A BFF per screen or per developer instead of per client experience.
- A BFF owned by a separate backend team, losing the main benefit.
- Sequential calls in the BFF where parallel calls would do.

5 Key terms

BFF: backend built for one client experience

Aggregation: combine several service calls into one response

Response shaping: trim and format data for a specific screen

General-purpose API: one API for all consumers

GraphQL: query language letting clients choose fields

Partial response: return what's available when one call fails

6 Interview questions

Q1. What is a backend-for-frontend? BASIC

A thin backend built for one client experience, owned by that frontend team, that aggregates shared services and returns data shaped for its screens.

Q2. How do a BFF and an API gateway relate? BASIC

They complement each other: the gateway handles edge concerns like TLS, auth and rate limits, then routes to the BFF that aggregates and shapes data.

Q3. Why use a BFF instead of one general gateway? SCENARIO

Each client gets an API shaped for its needs, aggregation happens server-side, and the frontend team ships its BFF independently; a general API bloats with every client's special cases.

Q4. Design the mobile home screen API. SCENARIO

A mobile BFF calls profile, recommendations, messaging and orders in parallel with timeouts, returns one ~2 KB payload, and degrades gracefully if one call fails.

Q5. What's the main risk of BFFs? TRICKY

Duplicated logic across BFFs that drifts apart; keep business rules in the domain services and BFFs thin.

Q6. When would you choose GraphQL over BFFs? TRICKY

When many clients need flexible field selection over the same data and you'd rather run one schema than many backends, accepting query-cost control and caching complexity.

Next: SD-L-009 · Service Mesh backendroadmap.com